```
!pip install nltk hmmlearn
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.13/dis
Collecting hmmlearn
  Downloading hmmlearn-0.3.3-cp313-cp313-manylinux_2_17_x86_64.manyl
Requirement already satisfied: click in /usr/local/lib/python3.13/di
Requirement already satisfied: joblib in /usr/local/lib/python3.13/d
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/pyt
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dis
Requirement already satisfied: numpy>=1.10 in /usr/local/lib/python3
Requirement already satisfied: scikit-learn!=0.22.0,>=0.16 in /usr/l
Requirement already satisfied: scipy>=0.19 in /usr/local/lib/python3
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/li
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/py
Downloading hmmlearn-0.3.3-cp313-cp313-manylinux_2_17_x86_64.manylir
Installing collected packages: hmmlearn
Successfully installed hmmlearn-0.3.3
```

```
import nltk
import numpy as np
import re
```

```
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.classify import NaiveBayesClassifier
```

```
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
documents = [
    ("This document contains the title and author name", "Header"),
    ("Introduction to Natural Language Processing", "Header"),
    ("Machine learning is a branch of artificial intelligence", "Bo
    ("NLP helps computers understand human language", "Body"),
    ("This research explains text classification techniques", "Body
    ("Page number 10 copyright 2026", "Footer"),
    ("All rights reserved", "Footer")
]
```

```
stop_words = set(stopwords.words('english'))
```

```
def preprocess(text):
    words = word_tokenize(text.lower())
    words = [
        word for word in words
        if word.isalpha() and word not in stop_words
    ]
    return words
```

```
def extract_features(text):
    words = preprocess(text)
    return {word: True for word in words}

# Step 6: Prepare training data
training_data = [
    (extract_features(text), label)
    for text, label in documents
]
```

```
classifier = NaiveBayesClassifier.train(training_data)
```

```
test_document = [
    "Introduction to Artificial Intelligence",
    "Natural Language Processing helps computers understand text",
    "Copyright 2026 All rights reserved"
]
```

```
print("DOCUMENT SEPARATION RESULTS\n")

for sentence in test_document:
    features = extract_features(sentence)
    prediction = classifier.classify(features)

    print("Text:", sentence)
    print("Predicted Section:", prediction)
    print()
```

```
DOCUMENT SEPARATION RESULTS

Text: Introduction to Artificial Intelligence
Predicted Section: Body

Text: Natural Language Processing helps computers understand text
Predicted Section: Body

Text: Copyright 2026 All rights reserved
Predicted Section: Footer
```